- By storing policies in a version control system, they can be versioned, rolled back, and audited.
- Policies can be tested using the same testing frameworks that developers already employ or custom frameworks that provide special features.
- Organizations can use the same tools they already use for continuous integration and deployment (CI/CD) to enforce integration testing and security reviews before policy changes go live.

Product managers, developers, or security teams can write policies that are treated as code. A general-purpose policy engine evaluates the policies to enforce access whenever a request is received.

Policy as Code is useful for situations where broad, reusable rules can be applied to the problem. For example, in a zero trust architecture, a policy might say, “Only employees on managed devices can access critical resources.” In a financial services application, a policy might say, “Only employees in the finance department whose training is up to date can issue purchase orders.” These kinds of rules are broadly applicable and can be combined to handle more complex scenarios.

Policy as Code use policies are written in a policy language. We’ll use the Cedar policy language for examples in this book. Cedar is discussed in detail in chapter 8. Here’s an example of a Cedar policy that forbids any action by anyone on resources in the category `CriticalResources` unless the device posture signal indicates they are using a device that is judged `Secure` with an overall security assessment greater than or equal to `80`.

```
forbid(
    principal,
    action,
    resource in Category::"CriticalResources"
) unless {
    context.device.security.risk == "Secure" &&
    context.device.security.assessment.overall >= 80
};
```

This policy uses code to clearly state the policy writer's intentions. This is a `forbid` policy, meaning that if the requirements of the policy are not met,